

Complemento Práctico: Dejando el Código Espagueti

Hasta ahora, escribíamos nuestras consultas SQL directamente en el main o mezcladas con la lógica del programa. Esto funciona para scripts pequeños, pero es una pesadilla para sistemas reales. ¿Qué pasa si mañana cambiamos MySQL por Oracle? ¿O si queremos agregar validaciones complejas antes de guardar?

En este cierre de materia, aprenderemos a estructurar una aplicación como lo hacen los profesionales: separando responsabilidades. Veremos cómo el DAO se encarga de hablar con la base de datos, el Service protege las reglas del negocio, y el Model transporta los datos entre ellos.

Esta arquitectura por capas no es solo teoría académica: es el estándar de la industria que utilizan empresas de todo el mundo para construir aplicaciones escalables, mantenibles y profesionales.



Contenido

Arquitectura por Capas

Comprende cómo separar responsabilidades entre las capas Model, DAO y Service para una estructura de aplicación profesional.

Seguridad y Buenas Prácticas

Aprende a proteger las aplicaciones de inyecciones SQL utilizando PreparedStatement y a gestionar transacciones correctamente.

Gestión de Datos

Domina el flujo de datos desde la interfaz de usuario hasta la base de datos, incluyendo el manejo de ResultSet y la gestión de conexiones.

Transacciones y Consistencia

Entiende el uso de commit, rollback y AutoCommit para garantizar la integridad de los datos.

El Problema del Código Desorganizado

¿Por qué evitar SQL en la UI?

Mezclar consultas con la interfaz crea aplicaciones imposibles de mantener

- Código duplicado en múltiples lugares
- Imposible cambiar de base de datos sin reescribir todo
- Las reglas de negocio se pierden entre líneas de SQL
- Testing unitario prácticamente imposible
- Cambios pequeños requieren tocar decenas de archivos

Imagina que tu aplicación crece y tienes 50 pantallas diferentes que guardan productos. Si la estructura de la tabla cambia, tendrías que modificar 50 lugares distintos. Con una arquitectura por capas, solo modificas un archivo: el DAO.

La separación de responsabilidades no es un capricho académico, es una necesidad práctica que te ahorrará semanas de trabajo y dolores de cabeza en proyectos reales. Cada capa tiene un propósito específico y conoce solo lo necesario para cumplir su función.

Arquitectura por Capas: La Visión General

Capa de Presentación (UI)

Interactúa con el usuario. Captura datos, muestra resultados. No conoce la base de datos.

Capa de Negocio (Service)

El cerebro de la aplicación. Valida, calcula, decide. Protege las reglas del dominio.

Capa de Datos (DAO)

El único lugar con SQL. Traduce entre objetos Java y tablas relacionales.

Esta separación crea fronteras claras entre responsabilidades. La UI no sabe cómo se guardan los datos. El Service no conoce detalles de SQL. El DAO no valida reglas de negocio. Cada componente es especialista en su tarea y puede evolucionar independientemente.

El Modelo (DTO): Objetos Mensajeros

El primer paso para ordenar la casa es dejar de pasar variables sueltas (String nombre, double precio) y empezar a pasar **Objetos Completos**. El **Modelo** o **DTO (Data Transfer Object)** es una clase simple que representa una tabla de la base de datos.

Un DTO es un POJO (Plain Old Java Object): no tiene lógica compleja, no hace cálculos, no valida. Su único trabajo es *transportar datos* entre las capas de la aplicación. Es como un sobre que lleva información de un lugar a otro.

❏ **Regla de Oro:** Los DTOs no contienen SQL, no abren conexiones, no validan reglas de negocio. Solo datos y sus getters/setters.

```
public class Producto {
    private Long id;
    private String nombre;
    private Double precio;
    private Integer cantidad;

    // Constructor vacío
    public Producto() {}

    // Constructor con parámetros
    public Producto(String nombre,
        Double precio,
        Integer cantidad) {
        this.nombre = nombre;
        this.precio = precio;
        this.cantidad = cantidad;
    }

    // Getters y Setters
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    // ... más getters/setters
}
```

El Patrón DAO: El Traductor Oficial

El **DAO (Data Access Object)** es el *único* lugar de toda la aplicación donde está permitido escribir SQL. Su responsabilidad es **traducir** entre el mundo de Objetos Java y el mundo Relacional de la base de datos. Es el traductor oficial que habla ambos idiomas.

La regla de oro: el resto del programa NO debe saber qué base de datos usamos, ni cómo se conecta, ni qué dialecto SQL escribimos. Por eso, el DAO suele implementar una **Interfaz** que define el contrato de operaciones disponibles, sin revelar los detalles de implementación.



```
// Interfaz: El "qué"
public interface ProductoDAO {
    void guardar(Producto p) throws SQLException;
    List<Producto> listar() throws SQLException;
    Producto buscarPorId(Long id) throws SQLException;
    void actualizar(Producto p) throws SQLException;
    void eliminar(Long id) throws SQLException;
}
```

```

public class ProductoDAOImpl implements ProductoDAO {

    @Override
    public void guardar(Producto p) throws SQLException {
        String sql = "INSERT INTO productos " +
            "(nombre, precio, cantidad) " +
            "VALUES (?, ?, ?)";

        try (Connection conn = ConexionDB.getConexion();
            PreparedStatement stmt = conn.prepareStatement(sql,
                Statement.RETURN_GENERATED_KEYS)) {

            stmt.setString(1, p.getNombre());
            stmt.setDouble(2, p.getPrecio());
            stmt.setInt(3, p.getCantidad());

            stmt.executeUpdate();

            // Recuperar ID generado
            ResultSet rs = stmt.getGeneratedKeys();
            if (rs.next()) {
                p.setId(rs.getLong(1));
            }
        }
    }

    @Override
    public List<Producto> listar() throws SQLException {
        List<Producto> productos = new ArrayList<>();
        String sql = "SELECT * FROM productos";

        try (Connection conn = ConexionDB.getConexion();
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(sql)) {

            while (rs.next()) {
                Producto p = new Producto();
                p.setId(rs.getLong("id"));
                p.setNombre(rs.getString("nombre"));
                p.setPrecio(rs.getDouble("precio"));
                p.setCantidad(rs.getInt("cantidad"));
                productos.add(p);
            }
        }
        return productos;
    }
}

```

Nota cómo todo el código SQL está aislado aquí. Si mañana cambiamos a PostgreSQL u Oracle, solo modificamos esta clase. El resto de la aplicación ni se entera del cambio.

La Capa de Servicio: El Cerebro del Negocio



Aquí es donde muchos estudiantes se confunden: "¿Por qué no llamo al DAO directamente desde el Main?" Porque el DAO es tonto; solo sabe guardar. No sabe si los datos son válidos, no conoce las reglas del dominio, no puede tomar decisiones de negocio.

El **Service** es el guardián. Es quien aplica las **Reglas de Negocio** antes de permitir que los datos lleguen a la base de datos. Es la inteligencia de la aplicación.



Validar

¿El precio es negativo? ¿El nombre está vacío? Lanza excepciones propias si algo está mal.



Orquestar

¿Necesito guardar en dos tablas? ¿Actualizar inventario? Coordina operaciones múltiples.



Delegar al DAO

Si todo está bien, le pasa la pelota al DAO para la persistencia real.

```
public class ProductoService {
    private ProductoDAO dao = new ProductoDAOImpl();

    public void registrarProducto(Producto p) throws Exception {
        // 1. Validaciones de Negocio
        if (p.getNombre() == null || p.getNombre().trim().isEmpty()) {
            throw new IllegalArgumentException(
                "El nombre del producto es obligatorio");
        }

        if (p.getPrecio() == null || p.getPrecio() < 0) {
            throw new IllegalArgumentException(
                "El precio no puede ser negativo");
        }

        if (p.getCantidad() == null || p.getCantidad() < 0) {
            throw new IllegalArgumentException(
                "La cantidad no puede ser negativa");
        }

        if (p.getCantidad() > 1000) {
            throw new IllegalArgumentException(
                "Exceso de stock: máximo 1000 unidades");
        }

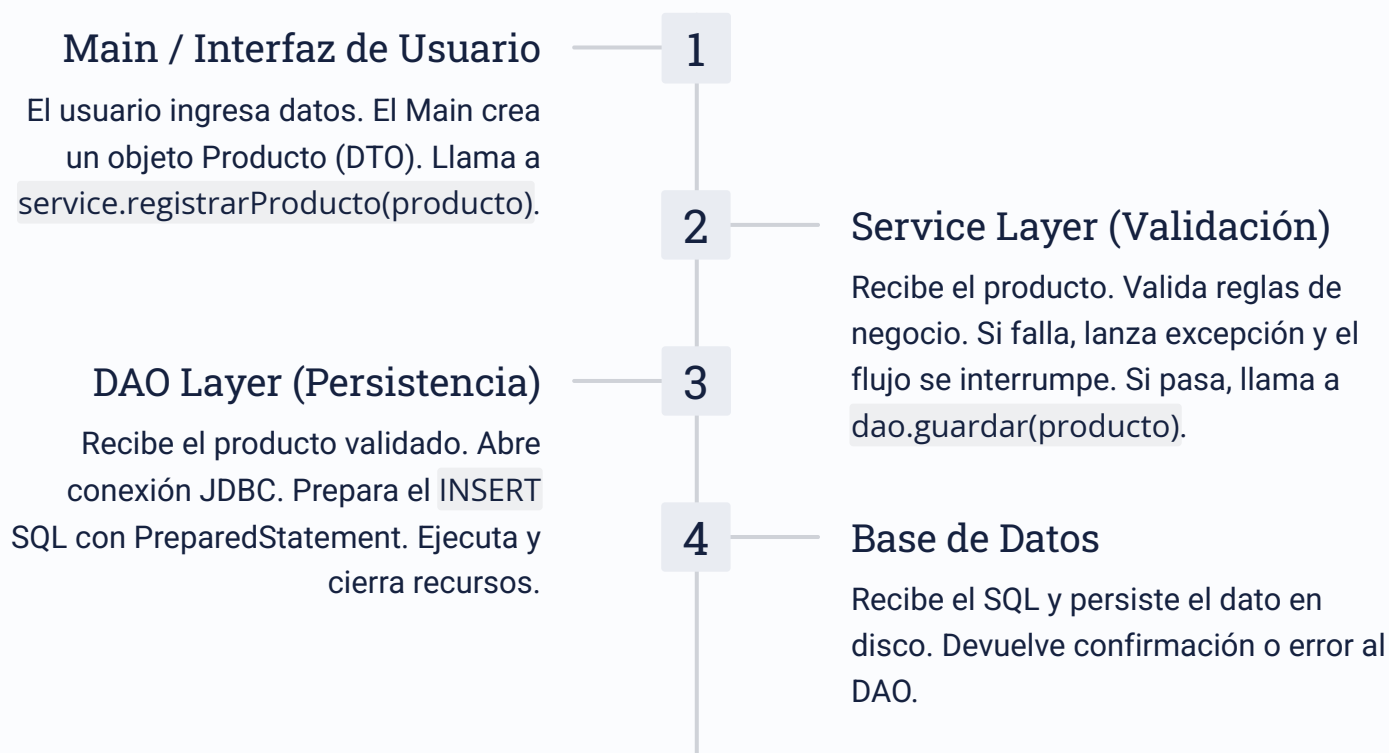
        // 2. Si todas las validaciones pasan, delegamos al DAO
        dao.guardar(p);
    }
}
```

El Flujo Completo: Trazabilidad del Dato

Visualicemos el camino completo de un Producto desde que el usuario lo crea hasta que se persiste en disco. Esta trazabilidad es fundamental para entender cómo interactúan las capas y por qué cada una es necesaria.



Este flujo unidireccional garantiza que los datos pasen por todos los controles necesarios antes de llegar a la base de datos. No hay atajos: la UI no puede saltarse el Service, y el Service no puede saltarse el DAO.



Ventajas de la Arquitectura por Capas



Mantenibilidad

Cambios localizados. Modificar la BD solo afecta al DAO. Cambiar validaciones solo toca el Service.



Testeabilidad

Cada capa puede probarse independientemente. Podemos crear un DAO falso para testear el Service sin base de datos real.



Reusabilidad

El mismo Service puede usarse desde una app web, una app de consola o una API REST sin cambios.



Trabajo en Equipo

Diferentes desarrolladores pueden trabajar en capas distintas simultáneamente sin conflictos.



Escalabilidad

Fácil agregar caché, pool de conexiones o cambiar a microservicios cuando el proyecto crece.



Seguridad

Las validaciones centralizadas previenen datos inconsistentes o maliciosos en la base de datos.

Cierre de Programación 2

¡Felicitaciones! Has completado el recorrido completo de Programación 2. Empezamos con variables primitivas y estructuras básicas, y terminamos construyendo una arquitectura empresarial por capas, conectada a bases de datos reales y protegida contra errores con un manejo profesional de excepciones.

POO Sólida

Herencia, Polimorfismo, Interfaces, Clases Abstractas y Genéricos

Robustez

Manejo profesional de Excepciones y gestión correcta de Recursos

Persistencia

JDBC, Transacciones, Patrón DAO y conexión con bases de datos reales

Arquitectura

Diseño modular, desacoplado y escalable con Service Layer

No solo aprendiste a programar: aprendiste a *diseñar software*. La diferencia entre escribir código que funciona y escribir código que perdura. Entre resolver un problema hoy y crear una solución que pueda evolucionar durante años.

Estás listo para enfrentar desafíos de desarrollo backend en el mundo real. Las empresas buscan desarrolladores que no solo escriban código, sino que entiendan arquitectura, separación de responsabilidades y buenas prácticas. Tienes esas herramientas. **¡Éxitos en el final y en tu carrera profesional!**